A Technology Vision for Quantitative Trading

Thomas Schmelzer, Jebel Quant Research — May 2026

Jebel Quant Research builds the tools and infrastructure for systematic trading teams — from data access and portfolio construction to live execution and repo management. This document sets out the thinking behind that work.

These are my personal views on how a quantitative trading platform should be built. I have spent two decades working across systematic hedge funds, high-frequency trading, family offices and sovereign wealth funds. The practices I describe in the early sections were perfectly reasonable at the time. Technology has moved on, and I think the way we build and run these platforms should move on too.

Before finance, I trained and qualified as a professional mechanic at AUDI. As a student I spent a summer assisting in their quality labs. When I reach for analogies about how precision work gets done — kitchens, assembly lines, shared standards — I am drawing on direct experience, not metaphor.

The Old Approach

For most of the industry's history, quantitative research followed a familiar pattern. A small group of researchers, typically mathematicians, physicists and statisticians, would develop trading strategies in MATLAB or Python. These environments suited the work well: easy to iterate on, rich in numerical libraries and close to the mathematical notation of the models. A researcher could move from an idea to a backtest in a matter of days.

The outputs were scripts: dense, clever and personal. Variable names made sense to their authors. Logic accumulated in layers over months or years. The code worked, and it carried real insight about markets.

Reinventing the Wheel

Because research lived in personal scripts, knowledge did not accumulate. Each researcher wrote their own version of the same basic components: moving averages, momentum signals, mean-reversion filters, portfolio construction routines. There was no shared library, no common vocabulary in code. Two researchers on the same team might implement the same idea with different conventions, different edge case handling, different numerical choices. When results diverged, it was hard to know why.

This also made it difficult to build on what had come before. Onboarding a new researcher often meant starting from scratch. Institutional knowledge lived in people rather than in code, and walked out the door when those people moved on.

There is a subtler problem too. Researchers working in isolation tend to gravitate towards strategies they can implement themselves. The limit is often their programming skills rather than their research ideas. A researcher with strong mathematical intuition but modest software engineering experience will keep returning to the same simple constructions, not because they are the best ideas available but because they are the ones within reach. A shared platform with well-built common tools raises that ceiling. The researcher's ambition is no longer bounded by what they can personally code from scratch.

The Handover Problem

When a strategy was ready for production, it was passed to a team of software engineers to reimplement in C++. The rationale made sense: C++ offered the performance, determinism and operational robustness that live trading required. But the process was expensive.

The handover was rarely clean. A MATLAB matrix operation that fit on one line might require careful memory management and numerical precision decisions in C++. Edge cases the script handled implicitly had to be made explicit. Bugs crept in during reimplementation and were hard to catch because the reference and production implementations diverged the moment the handover began.

The result was a slow pipeline. Research cycles were constrained by engineering capacity. Months could pass between an idea and a live strategy. Every change to a live strategy risked restarting the process.

Modern machine learning made this untenable. Libraries like PyTorch represent millions of engineering hours: automatic differentiation, GPU kernels, distributed training, a vast ecosystem of pretrained models. Reimplementing any meaningful fraction of that in C++ is not a project; it is a decade-long programme. Teams that held to the C++ mandate found themselves unable to use these tools, and fell behind those that did.

History has not been kind to the handover model. Across the industry, the pattern repeated itself: the central tools were never built. Teams focused on strategies and left the shared infrastructure as an afterthought. Essential tools — data access, portfolio construction, performance analytics, backtesting frameworks — were created independently by every team that needed them, in slightly different ways, with slightly different assumptions. The same wheel was reinvented dozens of times across the same organisation, and nobody had a complete picture of what existed or how reliable any of it was. The handover model produced not one coherent platform but a sprawling collection of overlapping partial solutions, each owned by whoever happened to write it.

There is a human cost to the handover model that rarely gets discussed. When something goes wrong in a live strategy, the handover creates a ready-made alibi for everyone involved. The researcher points to the

Python code and says it was correct. The engineer points to the C++ implementation and says it faithfully reproduced what was handed over. The operations team says they deployed exactly what they were given. Nobody is lying, and nobody fixes anything quickly. In a system built on a clean handover between separate groups, accountability diffuses precisely at the moment when it is most needed. The checkerboard structure is partly a response to this. When researchers and developers have built something together, they own it together.

The handover model can be made to work better. Shared interface definitions between research and engineering reduce ambiguity at the boundary. Strict versioning of the research artefact — the exact notebook, the exact data snapshot, the exact parameters — gives the engineering team something precise to reimplement rather than a moving target. Automated regression tests that compare research and production outputs on the same inputs catch divergence early, before it reaches live capital. These are real improvements and worth making. But they are patches. They reduce the cost of the translation step; they do not eliminate it. The fundamental problem is that translation step itself — the moment where one representation of an idea becomes another, and where something is always lost or changed in the crossing.

A New Direction

Our idea is built around a single environment for both research and production. Researchers express ideas in high-level terms and the platform carries them through to execution without a translation step.

The old handover model borrowed, implicitly, from an outdated idea of the factory: one group does its work and passes the output down the line. It is worth being precise about which factory. Henry Ford's assembly line was built on fixed interfaces, specialised stations and inspection at the end. A modern car factory looks nothing like this. Toyota's production system — quality checks at every station, workers empowered to stop the line, continuous feedback between stages — is far closer to what we advocate here than to anything Ford would recognise. The quant industry adopted a model that manufacturing itself had largely abandoned by the 1980s. Marcos Lopez de Prado has argued explicitly for the factory model in his work on the industrialisation of quantitative finance — and the model was put into practice at ADIA's Team Q, where I had the opportunity to observe its strengths and its costs firsthand.

The deeper problem is not that the factory model is wrong in general, but that it does not fit knowledge work at all. In a factory, the interface between stations can be fully specified in advance: a part has a known shape and tolerance before it arrives at the next station. In research and engineering, the interface is exactly what is hardest to define, and it changes as understanding develops. Treating it like a fixed handoff creates the appearance of process while undermining the collaboration that actually gets things done.

A more useful image is the professional kitchen. Not the frantic Saturday-night service of a Michelin-starred

restaurant — that version of the kitchen, with its noise and urgency, is the wrong picture. Think instead of the kitchen as atelier: calm, deliberate, precise. A place where skilled people work together with shared tools, each aware of what the others are doing, unhurried but never idle. A kitchen of that kind is not sequential. Every station is visible to every other. The head chef and the junior cook share the same space and the same standards. Quality is not inspected at the end; it is everyone's responsibility throughout. No serious dish leaves having passed through one pair of hands with no awareness of what came before or after.

A quant fund should work the same way. Researchers and developers in the same room, aware of each other's constraints, catching problems early. Quality is not a downstream function.

The kitchen analogy has a further implication. A chef is not expected to build the oven or construct the fridge. A professional kitchen assumes a working environment: reliable equipment, the right tools available. Asking a chef to fabricate their own appliances would produce worse food and worse appliances. The same applies here. Researchers and developers should not be building basic infrastructure from scratch: data pipelines, execution connectors, backtesting engines, monitoring tooling. That is what the platform provides. The team's energy belongs on the signals, the models, the risk framework. Everything else is the oven, and it should just work.

The mechanism that makes this real is containerisation. A container packages not just code but the entire runtime environment — libraries, language version, dependencies — so the environment a researcher uses to develop a strategy is identical to the one that runs live. Moving between research, backtesting and production is a matter of changing configuration, not rewriting code. The familiar complaint — "it works on my machine" — loses its meaning when every machine runs the same machine.

We organise the team in what we call a checkerboard structure rather than separating researchers and developers upstream and downstream. They sit together, alternate and collaborate continuously. A researcher working on a new signal works alongside the engineer responsible for the infrastructure that will run it. Knowledge moves in both directions. The result is code that is correct and deployable from the start.

In practice the boundary between researcher and developer is often blurry, and we think that is a good thing. The modern developer on this platform understands the models and contributes to the research process. The modern researcher writes production-quality code and takes responsibility for what they ship. We assume most researchers have strong development skills, and most developers have enough quantitative depth to engage seriously with the research.

Building the Kitchen

Building the platform means deciding what belongs in every kitchen regardless of what is being cooked. Several components are non-negotiable.

Data API. Access to clean, reliable data is the foundation. Every strategy, every backtest, every risk calculation depends on it. A well-designed data API handles the complexity of sourcing, normalising and versioning data across providers and asset classes so researchers can ask for what they need without worrying about how to fetch it. Getting this layer right is what everything else is built on.

Common strategy tooling. Many of the building blocks of quantitative strategies are not proprietary. Portfolio construction, signal combination, position sizing, transaction cost modelling are shared across the industry and across the team. They should be implemented once, tested thoroughly and available to everyone. A researcher building a new strategy should not be reimplementing a portfolio optimiser or a signal blending framework from scratch.

Performance analytics. Understanding why a strategy performed the way it did matters as much as the performance itself. A standard set of analytics, covering returns decomposition, drawdown analysis, factor attribution and cost accounting, means every strategy can be evaluated consistently. Results are comparable across strategies and over time rather than each researcher maintaining their own metrics.

Live monitoring. A strategy in production requires continuous observation: position and exposure tracking, P&L attribution, signal behaviour, execution quality and alerting when something moves outside expected bounds. Problems should be visible before they become costly.

Execution layer. Strategies communicate intent through a standardised API; the execution layer translates that intent into orders, handles broker connectivity and manages the operational complexity of running strategies at scale. This is part of the kitchen — something the team should not have to rebuild for every strategy. It is discussed in detail in the Live Trading section.

One principle applies across all of this. The kitchen must be built with researchers, not just for them. A platform designed only by engineers, however capable, risks solving the wrong problems. Researchers know what data they actually need, how they think about portfolio construction, what a useful performance report looks like and what slows their work down. That knowledge needs to be in the room when the platform is being built.

In practice, the kitchen and the first strategies are often built in parallel. The team cannot wait for a complete platform before starting research, and waiting would be the wrong instinct anyway: building infrastructure in isolation, without real strategies pushing against it, tends to produce the wrong infrastructure. The feedback

loop between strategy development and platform development is valuable and should not be broken.

That said, we do recommend establishing a minimal set of common tools before the first strategies are implemented. At minimum this means a working data API, a basic portfolio construction library and a consistent project structure enforced by Rhiza — a tool that keeps every strategy repository aligned with a common template. Without these in place, the first strategies will each invent their own solutions, and unpicking that fragmentation later is costly. A small shared foundation built early pays back many times over.

Keeping the Platform Consistent

With the kitchen in place, strategies can be built in earnest. Each strategy lives in its own repository, but left unmanaged a collection of strategy repos quickly becomes a zoo. CI workflows diverge. Python versions drift. Linting configs split. A security fix lands in one repo and is missed by the rest. The same fragmentation that plagued the old world of personal scripts reappears at the infrastructure level.

Rhiza was built to address this directly. Rather than generating project scaffolding once and walking away, it keeps every strategy repository continuously aligned with the platform's canonical standards. When the central template changes — a new CI workflow, an updated linting config, a change to the containerisation setup — Rhiza opens a pull request in each downstream repo with a clear diff of what changed. Owners review, adapt where needed and merge. Nothing is forced and nothing is missed.

This is the infrastructure equivalent of shared strategy tooling. Researchers should not reimplement portfolio construction from scratch, and developers should not be manually maintaining CI pipelines in every repo. Rhiza keeps the scaffolding consistent so the team's attention stays on what is inside it.

Backtesting

A backtest is only useful if it is honest. The history of quantitative finance is littered with strategies that looked compelling on paper and disappointed in production, and the gap is rarely explained by bad ideas. It is almost always explained by a backtest that was, in some subtle way, too optimistic.

The most common culprit is look-ahead bias: the strategy had access to information during the backtest that it could not have had at the time. This can happen through data that has been revised after the fact, through target variables that leak future information into the features, or simply through a timestamp that is off by one bar. The platform enforces strict point-in-time data semantics. Every data query is anchored to a historical timestamp, and the data API makes it structurally difficult to request information that would not have been available at that point.

Transaction costs are the second place where backtests mislead. A strategy that ignores market impact, bid-

ask spreads and borrow costs can look highly profitable while being economically meaningless at any realistic scale. The platform models transaction costs explicitly, and the cost model is calibrated against real execution data where available. A researcher should be able to see what a strategy's net performance looks like under conservative, realistic and optimistic cost assumptions before it is taken seriously.

Overfitting is harder to guard against because it is partly a discipline problem rather than a tooling problem. The platform supports walk-forward analysis and out-of-sample evaluation as standard, making it easy to separate the in-sample period used for development from the out-of-sample period used for evaluation. But no tool prevents a researcher from repeatedly tweaking a strategy until the out-of-sample period looks good too. The culture of the team, and the scrutiny applied during strategy review, matters as much as the infrastructure.

A backtest that passes these tests is not a guarantee. Markets change, and a strategy that worked for ten years may stop working. The backtesting framework provides evidence, not certainty. The team should treat strong backtest results with interest and some scepticism in equal measure.

Live Trading

When a strategy goes live, the cost of any discrepancy between research and production becomes real and immediate. A backtest that behaves differently from the live system, because of an environment mismatch or a parameter misconfigured during deployment, can produce losses that no amount of prior testing would have flagged. Closing this gap is a core design requirement, not a convenience.

Proximity is the answer. The container a researcher uses to develop and backtest a strategy is the same container that runs in production. There is no reimplementation, no port, no translation step where something can silently go wrong.

What changes between environments is the configuration, not the code. The container is a fixed, versioned artifact. Configuration files tell it where to find data, which parameters to use, what risk limits to respect and which execution venue to connect to. Moving a strategy from backtesting to paper trading to live means changing the configuration it runs against. The container has no knowledge of which environment it is in. It reads its configuration and runs.

This separation of code from configuration is what makes promotion between environments safe and auditable. Every configuration file is versioned. Every deployment is a known container image combined with a known configuration state. If something goes wrong in production, the team can reconstruct exactly what was running and with what parameters. Rolling back is a configuration change, not an emergency deployment.

Each strategy is implemented as a service with a standardised API. This is a deliberate architectural choice: once the strategy exposes a clean interface, execution logic and scaling concerns can be handled by a separate

layer entirely. The strategy itself does not need to know how many instances are running, how orders are routed, or how load is distributed. That separation keeps the strategy code focused on what it is actually good at — generating signals and expressing intent — while the infrastructure layer handles the operational complexity of running it at scale.

Prime broker connectivity. The strategy never communicates with the outside world directly. It expresses intent through its API — buy this instrument, in this quantity, with this urgency — and the platform's execution layer, part of the kitchen, handles everything else: translation into FIX or a proprietary protocol, order routing, fill reconciliation, position and margin queries with the prime broker. This boundary is among the most consequential in the system. Errors here are not silent; they are immediate and financial. Keeping the strategy clear of that complexity is not just good architecture — it is what makes the system safe to operate.

Switching brokers or adding a new venue is a configuration change to the execution layer. The strategy is unaffected. In backtesting and paper trading the same interface is present, backed by a simulated fill engine rather than a live connection. The strategy code is identical across all environments. The broker, like everything else, is a detail the kitchen absorbs so the strategy does not have to.

Risk Management

Risk management is not a feature that gets added at the end. It is a layer that runs through the entire platform, from the moment a strategy is being designed to every order it places in production.

At the research stage, the platform provides tools for understanding the risk profile of a strategy before it goes anywhere near live capital. This means exposure analysis across factors, asset classes and geographies, as well as realistic stress testing against historical regimes. A strategy that looks attractive on raw returns but concentrates risk in ways the researcher has not examined is not ready, and the platform should make that visible early.

At the point of deployment, pre-trade risk checks sit between the strategy's intent and the execution layer. Position limits, notional limits, sector and instrument concentration limits, and maximum order sizes are enforced before any order leaves the system. These limits are, like everything else, configuration. They can be tightened or loosened without touching the strategy code, and every change is versioned and auditable.

In production, the platform monitors risk continuously. Drawdown limits trigger alerts and, if configured, automatic position reduction or a full halt. Gross and net exposure are tracked in real time against defined thresholds. If a strategy begins behaving in a way that is inconsistent with its historical risk profile, the monitoring layer surfaces that before it becomes a problem.

The kill switch is a first-class platform concept. Every live strategy can be stopped cleanly and immediately,

positions can be unwound in an orderly way, and the system returns to a known state. This is not an emergency procedure bolted on as an afterthought. It is something the team tests regularly, the same way a kitchen tests its fire procedures.

Conclusion

The problems described in this document are not technical failures. They are organisational ones. The handover model, the personal scripts, the reinvented wheels — none of these happened because teams lacked talent or ambition. They happened because the structures in place made sharing hard, translation inevitable and accountability diffuse. Better tooling alone does not fix that. The platform has to be accompanied by a different way of working.

What this document argues for is not a specific technology stack but a set of principles: shared environment over translation, quality at every stage over inspection at the end, common infrastructure over individual reinvention, accountability through shared ownership over alibi through separation. The specific tools — containers, Rhiza, a standardised execution API — are expressions of those principles, not the principles themselves. A team that internalises the principles will make good decisions about the tools. A team that adopts the tools without the principles will find ways to recreate the old problems inside the new infrastructure.

The edge in quantitative trading is scarce and hard to find. The platform exists to make sure that the search for it is not cluttered by problems that have already been solved.

Appendix: Jebel Quant Research

Jebel Quant Research develops the tools and infrastructure that the platform described here is built on. The work spans several areas.

Project infrastructure. Rhiza is the first publicly available tool from Jebel Quant Research. It solves the repo zoo problem for Python-heavy organisations by keeping project scaffolding continuously aligned across many repositories through a pull request based sync mechanism. It is already in use beyond Jebel Quant, including at Stanford's CVXGRP and Janus Henderson. Relevant repos: rhiza, rhiza-cli, rhiza-tools, rhiza-hooks, rhiza-education.

Data access. A clean, versioned API into market data is foundational to everything else. Jebel Quant Research has developed tooling for sourcing, normalising and serving data across asset classes, with a consistent interface that works identically in research and production.

Portfolio construction. Drawing on work developed in collaboration with Stephen Boyd's group at Stanford and informed by co-authored research with Ron Kahn, Jebel Quant Research has built portfolio construction tools grounded in convex optimisation. These cover mean-variance optimisation, transaction cost aware rebalancing and risk-constrained allocation. Relevant repos: linalg, basanos.

Signal combination and performance analytics. Tools for combining signals from multiple sources, evaluating strategy performance consistently and attributing returns across factors and time periods. The goal is a shared analytical vocabulary across the team rather than each researcher maintaining their own metrics. Relevant repos: jquantstats.

Live trading infrastructure. The container-based deployment model, configuration management framework and prime broker connectivity layer described in this document are products of Jebel Quant Research. They are designed to be reusable across strategies and, where appropriate, across organisations.

Further Reading

Marcos Lopez de Prado — *Advances in Financial Machine Learning* (Wiley, 2018). The assembly line model for quantitative research is discussed in Section 1.3. Lopez de Prado argues for a factory-style division of labour; this document argues for a different conclusion from a shared diagnosis. A working paper version is available at papers.ssrn.com/sol3/papers.cfm?abstract_id=3104847.